Design Decisions Q&A (corrected)
1. General Architecture
Main system blocks:
•ARM Cortex-A9 (HPS) running minimal Linux — file/metadata layer
•NIOS II soft core — real-time control layer
•Custom FPGA peripherals — streaming, timing, I/O
•Shared memory bridge between HPS and NIOS II
•External: SD card (storage), VGA monitor, 7-seg displays, buttons, switches, audio amplifier + speakers
Main processor: Both, with distinct roles. NIOS II is the **real-time conductor** — it owns the state machine, MMIO, and interrupts. The HPS is a **co-processor for file I/O** — it only reads the SD card, parses WAV files, and extracts metadata. NIOS II drives all user-facing behavior.
Hardware side (FPGA fabric):
•Interrupt controller (debounced button edges, one IRQ per press)
•Switches decoder + filter display chain (combinational, switch → DSP and switch → 7-seg)
•Timer module (resume/pause/reset commands, drives MM:SS 7-segs directly)
•16-bit → 4x 7-seg decoder for the timer output
•VGA drawing module + VGA signal module
•Custom FIFO sampler / audio streaming module (holds playback bookmark across pause/resume)
•DSP unit with the three filters
•I/O memory module (MMIO bus)
•NIOS II on-chip memory + JTAG
Software side:
•Linux on ARM: SD driver, FAT filesystem, WAV parser, metadata extractor, audio chunk loader
•NIOS II bare-metal: state machine, ISRs, MMIO drivers, song-pointer/linked-list logic, VGA buffer updates
Software control logic responsibilities (NIOS II):
•Track current state (Stopped/Playing/Paused) and current song index
•Handle button ISRs and dispatch state transitions
•**Issue resume/pause/reset commands to the timer module on every state change**
•**Issue resume/pause/reset commands to the FIFO sampler on every state change**
•Refill the audio FIFO when it signals it's hungry
•Update VGA metadata on state changes
•Request next/prev song data from the ARM
FPGA streaming/timing logic responsibilities:
•Generate sample clock at the song's configured sample rate (8/16/44.1 kHz)
•Pull samples from FIFO at exactly that rate, no jitter
•Hold the playback bookmark when paused, resume cleanly from it
•Apply the selected filter
•Drive the audio codec
•**Timer module** advance MM:SS on its own internal clock while resumed, freeze on pause, zero on reset
•Refresh 7-seg displays and VGA signal continuously

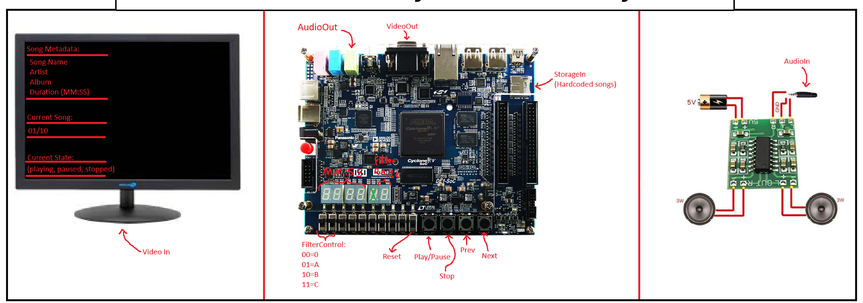
2. Hardware/Software Boundary
Data that crosses the boundary:
•ARM → NIOS II (shared RAM): audio sample blocks, metadata strings, song count
•NIOS II → audio FIFO (MMIO): audio samples, sample rate config, resume/pause/reset command
•Audio FIFO → NIOS II (IRQ or polled flag): "almost empty" / "needs refill" status
•NIOS II → timer (MMIO): resume / pause / reset command — that's it, no read-back
•NIOS II → VGA module (MMIO): metadata text, state, song index
•Interrupt controller → NIOS II: button ID + edge flag
•Switches → DSP and switch-display 7-seg: hardware-only, never touches software
Sample buffer/FIFO? Yes, mandatory. Software cannot meet sample-accurate timing for 44.1 kHz audio. The FIFO absorbs software jitter and decouples CPU loop rate from audio sample rate.
Who controls audio streaming rate/timing? The FPGA. The FIFO drains at a hardware-generated sample clock derived from the configured rate. Software just keeps the FIFO non-empty.
Push or pull? Software pushes, hardware pulls. NIOS II pushes blocks of samples into the FIFO whenever there's room; hardware pulls one sample per sample-clock tick out to the DSP. FIFO raises an IRQ (or NIOS II polls a low-watermark flag) when it needs more. Producer-consumer pattern — the only sane choice for the latency budget.

3. Audio and Storage Decisions
Format: WAV (PCM, uncompressed) — required by spec. No decoder needed, just header parsing.
Bit depth: 16-bit signed PCM — required by spec.
Mono or stereo? Recommend **stereo:**
•The PAM8403 amplifier shown in your diagram is stereo with L/R speakers
•The 3.5 mm jack is stereo
•WAV stereo files are standard
•FIFO width just becomes 32 bits (16L + 16R)
Mono is acceptable if you want to halve memory bandwidth and simplify the DSP, but you lose the speaker setup.
File storage: WAV files in a FAT32 partition on the micro-SD card. At least 10 songs per spec. Metadata from the WAV LIST/INFO chunk, with filename fallback.
SD card access: Through the HPS's built-in SD/MMC controller (dedicated silicon on the Cyclone V SoC, not something you build). Linux mounts it as a block device; standard fopen/fread in your ARM userspace program.
SD on HPS or FPGA? HPS / software. Building an SD controller + FAT32 driver in FPGA fabric is months of effort for zero gain — the HPS already has both. This is one of the clearest co-design decisions in the project.

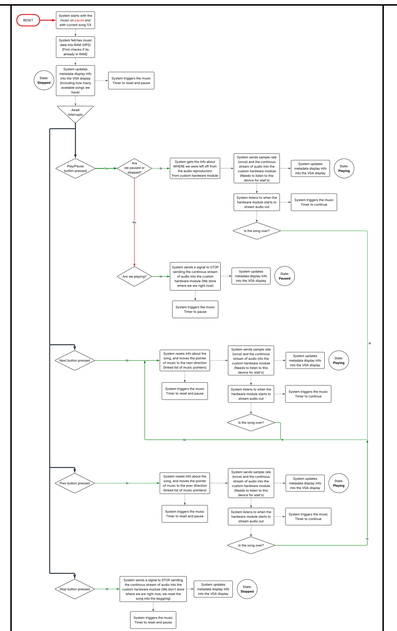
4. Filters, Display, and Custom Hardware
Filters (one from each pair, per spec):
•**FIR low-pass** — easier to design, inherently stable, linear phase. Recommended over IIR high-pass.
•**Band-pass** — straightforward as a single FIR with a band-pass kernel.
•**3-band equalizer** — more useful and audibly impressive than reverb, and reverb needs delay-line memory that complicates things.
Plus the no-filter passthrough = 4 modes, matching the 2-bit switch input.
Design filters ourselves or use IP? Hybrid:
•**Coefficients:** compute offline in Python/MATLAB (scipy.signal.firwin) — standard practice.
•**Filter hardware:** write the MAC pipeline yourself in Verilog/VHDL. It's the educational core of the DSP requirement and not much code (FIR = shift register + multiply-accumulate).
•**Audio codec interface:** use Altera's University Program Audio IP for the WM8731 — building that from scratch is out of scope, and the spec explicitly points you to the Platform Designer Audio IP.
So: codec IP from Altera, filter logic custom.
Required custom hardware module: The **Audio Streaming / FIFO Sampler module** is the natural choice — it cleanly fits the "control / status / data registers, MMIO-accessible" requirement:
•*Control registers:* resume, pause, reset, sample-rate select
•*Status registers:* FIFO level, almost-empty flag, underrun flag, playing/idle, current bookmark position
•*Data register:* sample push port
The timer module is a secondary candidate but simpler (just 3 commands and an internal counter). The FIFO/streamer is the more defensible "custom hardware" deliverable.
Visual interfaces:
•**VGA monitor:** metadata (song name, artist, album, duration), current song X/N, current state
•**4x 7-seg:** elapsed time MM:SS, driven directly by the timer module
•**1x 7-seg:** active filter (0/A/B/C), driven combinationally from the switches
•**LEDs (optional):** state indicators for quick debugging

5. Validation and First Prototype
Validating HW/SW communication:
• Define the **MMIO register map first**, before either team writes anything. This is the contract.
• Use JTAG/System Console to poke registers directly from a host PC — confirms each peripheral works without any NIOS II code. Especially useful for the timer: write resume/pause/reset over JTAG, watch the 7-segs.
• On NIOS II side, write a "register echo" program: read every status register, write to every control register, print over JTAG UART.
• For audio: push a known waveform (sine table) from NIOS II into the FIFO; verify on oscilloscope/headphones before involving the ARM.
• For ARM↔NIOS II: start with a fixed test pattern in shared RAM written by ARM, read and printed by NIOS II.
Independent module testing:
• Hardware modules: ModelSim/Questa testbenches per module (FIFO sampler, timer, DSP, VGA drawer, interrupt controller). Each passes simulation before going on the board. For the timer specifically: assert resume/pause/reset commands in sequence and verify the internal counter behaves correctly.
• *On-board hardware checks:* SignalTap II logic analyzer to capture internal signals during real operation.
• *Software stubs:* NIOS II code tested against a "fake" MMIO region (just on-chip RAM) before hardware is ready. ARM code tested on a Linux PC against the same SD card before touching the board.
• *Filter validation:* feed a known input (impulse, sine sweep) through the DSP in simulation and compare to a MATLAB reference.
• *Integration order:* NIOS II + interrupts + filter 7-seg first (cheapest feedback loop) → timer module (visible on 7-segs, easy to verify) → FIFO sampler with a synthetic sine source → VGA → ARM-fed real audio → filters → switches wired to DSP.
Golden rule: **never integrate two untested things at once.** Always add one module at a time to a known-working base.

General View of System Functionality

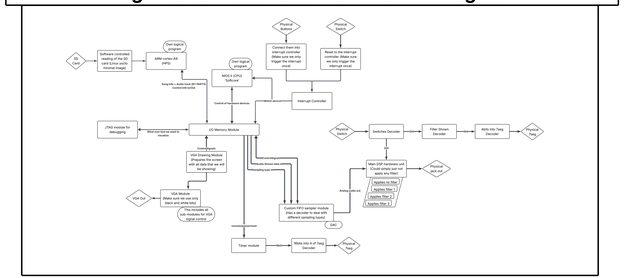


Flowchart of the general software system



Software Flow (Image 3 — The Flowchart)
The NIOS II runs an **interrupt-driven state machine** with three states: **Stopped**, **Playing**, **Paused**.
On reset:
1. Initialize to *Stopped*, current song = 1/N.
2. Ask the ARM to fetch song 1's data into shared RAM (skip if already cached).
3.Update VGA with metadata, reset timer.
4.Wait for interrupts.
Button handlers (each fires from the interrupt controller):
• **Play/Pause** — toggles based on current state:
  ◦ If Paused/Stopped → resume from saved position, start FIFO streaming, start timer → *Playing*.
  ◦ If Playing → tell FIFO to stop, save position, pause timer → *Paused*.
• **Next / Prev** — advance the song pointer (linked list of songs), reset position, reset timer, start streaming the new song → *Playing*.
• **Stop** — stop FIFO, reset position to song start, reset timer → *Stopped*.
• **Song-ends detection** — when the current song's audio is exhausted, automatically trigger the Next flow.
Every state change updates the VGA metadata block so the user always sees the current state.

High-Level Hardware Architecture Diagram



Hardware Architecture
Think of the FPGA fabric as a **bus of custom peripherals** orbiting around the NIOS II, with the ARM HPS sitting off to the side as a "file server."
Two brains, clearly split:
• **ARM Cortex-A9 (HPS) — the librarian.** Runs a minimal Linux image. Its only job is reading the SD card, parsing WAV files, extracting metadata, and shipping audio data in chunks to the NIOS II. It does not touch any real-time hardware.
• **NIOS II (soft core) — the conductor.** Runs bare-metal C with zero OS overhead. Coordinates everything: receives audio from the ARM, pushes it to the audio hardware, reacts to button interrupts, updates the VGA, and manages state.
The peripherals on the NIOS II side, grouped by role:
1. Input path (user controls):
• *Interrupt Controller* — sits between the physical buttons and the NIOS II. Debounces and ensures each press fires exactly one interrupt.
• *Switches Decoder* — reads the 2-bit filter switch, sends one path to the DSP unit (to actually apply the filter) and another path through a *Filter Display Decoder* → *7-seg Decoder* → physical 7-seg (to show which filter is active). This path is purely combinational, so filter switching is near-instant.
2. Audio output path:
• *Custom FIFO Sampler Module* — the heart of the audio engine. NIOS II writes audio samples into it; the FIFO drains them at the song's sample rate (8/16/44.1 kHz). This decouples software timing from real-time playback. NIOS II controls it via MMIO (resume/pause/reset, sample rate config).
• *Main DSP Unit* — receives the audio stream from the FIFO, applies the selected filter (or none), and outputs to the audio codec → physical jack.
3. Visual output path:
• *Timer Module* — counts elapsed playback time, controlled by NIOS II (start/pause/reset via MMIO). Output goes through a *16-bit → 7-seg Decoder* to four physical 7-segs showing MM:SS.
• *VGA Drawing Module* — NIOS II writes the metadata strings/state into its MMIO region; the module renders them to the VGA framebuffer and the *VGA Module* drives the actual VGA signal.
4. Glue:
• *I/O Memory Module* — the MMIO bus connecting NIOS II to all of the above.
• *On-chip memory* for NIOS II program/data.
• *JTAG* for debugging.
Communication between the two CPUs: ARM sends song info + audio data (in chunks) to NIOS II through shared memory or a mailbox. NIOS II sends control requests back (e.g., "give me the next song").

Why This Split Works
• **Hard timing stays in hardware:** sample-accurate audio (FIFO + DSP), time display (timer module), and filter switching (combinational logic) all bypass software latency. Filter switch + 100 ms and buttons + 200 ms are trivial because the critical paths don't touch the CPU.
• **Heavy lifting stays on the ARM:** filesystem, WAV parsing, and metadata extraction — things that need an OS and lots of memory — happen on the Cortex-A9 under Linux.
• **Real-time control stays on NIOS II:** bare-metal, deterministic, pointer-driven, talking only to MMIO peripherals.
• **Teams can work in parallel:** the MMIO contract (register map) is the interface. Hardware team builds and tests peripherals against that map; software team writes against the same map using stubs. They integrate at the boundary.